

Generative AI for Test Driven Development: Preliminary Results

Moritz Mock(✉), Jorge Melegati, and Barbara Russo

Free University of Bozen-Bolzano, 39100 Bolzano, Italy
{momock,jorge.melegati,brusso}@unibz.it

Abstract. Test Driven Development (TDD) is one of the major practices of Extreme Programming for which incremental testing and refactoring trigger the code development. TDD has limited adoption in the industry, as it requires more code to be developed and experienced developers. Generative AI (GenAI) may reduce the extra effort imposed by TDD. In this work, we introduce an approach to automatize TDD by embracing GenAI either in a collaborative interaction pattern in which developers create tests and supervise the AI generation during each iteration or a fully-automated pattern in which developers only supervise the AI generation at the end of the iterations. We run an exploratory experiment with ChatGPT in which the interaction patterns are compared with the non-AI TDD regarding test and code quality and development speed. Overall, we found that, for our experiment and settings, GenAI can be efficiently used in TDD, but it requires supervision of the quality of the produced code. In some cases, it can even mislead non-expert developers and propose solutions just for the sake of the query.

Keywords: AI4SE · Test Driven Development · Generative AI

1 Introduction

Test-driven development (TDD) is one of the major practices in Extreme Programming (XP) [1]. Nevertheless, its effectiveness is still controversial [7,9]. The major strength of TDD lies in its ability to deliver high-quality code through the granularity and uniformity of development [6]. To be effective, TDD developers must have a strong command of the practice and experience in development [3]. To facilitate its adoption, automation can be an option, and generative artificial intelligence (GenAI) tools, such as GitHub's Copilot[1] and OpenAI's ChatGPT[2], can be helpful [5]. In particular, recent literature has shown promising results in software testing. For instance, Piya and Sullivan [13] proposed an approach in which a test suite is fed to ChatGPT, and prompts are generated accordingly upon test failures. Liang *et al.* [11] have further shown that the use of GenAI

[1] https://copilot.microsoft.com.
[2] https://chat.openai.com.

© The Author(s) 2025
L. Marchesi et al. (Eds.): XP 2024 Workshops, LNBIP 524, pp. 24–32, 2025.
https://doi.org/10.1007/978-3-031-72781-8_3

can speed up testing. However, the quality of the generated tests and code and the role of the developers are still under discussion [11].

In this work, we explore the use of GenAI in automating TDD and reflect on the role of the developer. We then perform an exploratory experiment with five developers to assess the effectiveness of our method and compare it with non-AI TDD.

2 Methodology

Our goal is to automatize the TDD process with GenAI, exploring which minimal knowledge is needed in each iteration and which kind of role AIs and developers may have. To this aim, we developed a threefold methodology: first, we defined a workflow to automate the TDD process with GenAI, then we identified interaction patterns between developers and GenAIs supporting different types of automation activities of the workflow. Finally, we implemented a tool automatizing the workflow according to the interaction patterns and then performed an experiment to compare them. To design the workflow, we first identified the type of information that is handled in an iteration of a TDD process. The information includes the context, the feature to be developed, the test and production code, and the execution log that was eventually output in the previous iteration. To obtain coherent answers, we queried ChatGPT a few times with different types of prompts. The major challenge here is to obtain an incremental output. We do not want ChatGPT to generate the code for the feature in one shot as we are not implementing Test First [1]. Thus, we first automated the query process by implementing a Python script that leverages OpenAI's API to use ChatGPT as GenAI. We employed the model *gpt-3.5-turbo-16k*, which can have as context up to 16k tokens. We have also explored different ways of querying the ChatGPT: (i) not mentioning the testing task at all and retrieving each message as a stand-alone, (ii) including the output of the previous query as input for the next query, or (iii) querying with all data in (i) and (ii). Based on our observation, scenario (ii), in which we send the result of the last output, is the best one. In scenario (i), ChatGPT struggled to grasp the task, and in scenario (iii), it got confused. In the first attempts to query ChatGPT, we also observed that it had the tendency to produce the complete solution instead of performing incremental steps. To prevent from doing so, we added the sentence *"stub and drivers to develop the first barely minimal test and production code"*. From the second iteration onwards, the additional phrase *"Keep the existing tests"* at the beginning needed to be added so that the existing tests were not lost. In the end, we were able to formulate the following prompts:

- First iteration: Use the Assertion First pattern in TDD and stubs and drivers to develop the first barely minimal test and production code for the feature $\langle feature\ description \rangle$ with input $\langle names \rangle$ and $\langle values \rangle$ and expected output $\langle values \rangle$
- Intermediate iteration: Keep the existing tests and run the next iteration of TDD to develop the barely minimal test and production code

– Final iteration: Refactor the code.

A second step in our methodology consists of defining the role of the developers in the TDD process automated with GenAIs. To this aim, we defined three collaboration patterns: collaborative, fully-automated, and non-automated. In the *collaborative pattern*, we introduce an interaction between the human developer and the AI, in which the developer is in charge of writing the test code and modifying any test or production code generated by the AI before passing it again to the AI. Then, the AI generates the production code. The *fully-automated pattern* automates both steps. The developer only verifies the quality of the produced code at refactoring. The *non-automated pattern* does not involve any AI. Finally, we evaluated and compared the patterns with five practitioners who had experience with Python and TDD: three used the collaborative pattern, and two used the non-automated one. All have received the same initial exercise:

The goal of this experiment is to develop in Python the following feature: Develop a class TextFormatter that takes arbitrary words and horizontally center them into a line. The class TextFormatter shall have three functions. The first is called setLineWidth and sets the length of the line. The second function receives a single word and returns the word in the center of the line. The third function receives two words and centers the two words in the line. To develop it you will use Test Driven Development and, in particular, assertion first.

All participants were allowed to consult any source they liked. We recorded the screen while the participants performed their exercise. At the end of the task, they filled out a brief questionnaire. We compared the results in terms of the number of test functions, number of assertions, test LOC, code LOC, and time to complete the task. We also inspected their test, code, and logs (in case of automation) and qualitatively evaluated the quality of what has been produced. We further collected feedback from the participants. The code produced in the experiment can be found at https://github.com/moritzmock/AI4TDD.

3 Results

RQ1. Can generative AI be used to automate TDD? To answer this question, we designed two workflows, one implementing the collaborative pattern and the other fully-automated. We further implemented Python scripts that actuate them. Figure 1 illustrates the workflow for the fully-automated pattern: the activities with the AI symbol are performed by the AI model, and the execution of each activity in the workflow is automated by our scripts. The note boxes show the input needed for the next action: the prompts as described in Sect. 2 and the type of data to pass to the next activity. From the response of the AI, the production code is automatically integrated, and the test suite is launched. If the AI is not able to write code that fulfils the test case(s), the prompt is resent up to five times. In the collaborative pattern, the workflow is the same but different in its execution. Firstly, activities ① and ② are executed by the developer. Secondly, the developer can modify the input passed to the AI

in activity ③. The red text in the note boxes indicates the part of the output of the previous activity that the developer can modify. The rest is the same as in the fully-automated pattern.

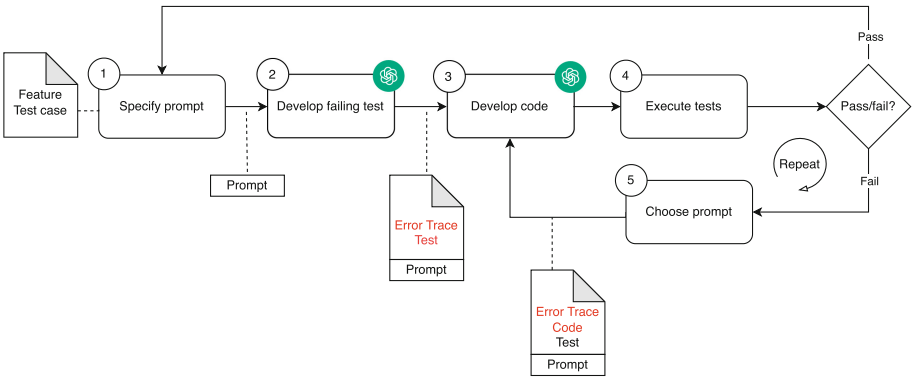


Fig. 1. Fully-automated pattern

In our experiment, we executed the two interaction patterns with five developers, whereas the authors launched the fully-automated pattern. In all cases, the experiment's task was successfully completed. The final test and production code, the process log, the screen recording, and a short feedback from the participants were collected.

RQ2. What kind of interaction model between generative AI and human developers is more promising? For our exploratory experiment, we selected five developers located in two different countries with some knowledge of the programming language and experience with TDD, as shown in Table 1.

Table 1. Demographics of the developers

	TDD Experience	Python Experience	Role	Interaction Scenario
P1	>3 years	<1 year	Software developer	collaborative
P2	1–3 years	>3 years	Data scientist	collaborative
P3	1–3 years	>3 years	Software developer	non-automated
P4	<1 year	<1 years	Software engineering student	collaborative
P5	1–3 years	<1 years	Software developer	non-automated

Three followed the collaborative pattern, and two followed the non-automated one. We compared the results of the experiment in terms of the metrics defined in Sect. 2. Table 2 reports the results for the five participants P1–P5 and the fully-automated pattern, F1. For F1, the authors launched the task a few times. The first times were used to learn the type of prompts needed to

automate TDD iteratively, and the last one was executed to compare the results of the fully-automated with the ones of the other patterns. Table 2 reports the value for the last run. It is worth noticing that the AI acts as a tester and developer, so we are able to log the activities of both. In some iterations, the developer implemented the code with no interaction with a tester, as may happen in real cases.

Table 2. Evaluation of the results for the participants P1, P2, and P3 and the fully-automated one F1. # of LOC does not contain blank lines.

	# test functions	# assertions	test LOC	code LOC	Time to complete	# iterations
P1	7	7	34	25	30 min	32
P2	3	9	19	19	30 min	49
P3	11	17	69	31	35 min	NA
P4	4	4	16	12	40 min	44
P5	3	3	16	14	40 min	NA
F1	1	3	14	17	12 min	8

Test Coverage and Code Inspection. All participants managed to complete the tasks within the time limit of 40 min. They created different sets of tests and considered different edge cases. P1 did not develop tests for all the valid inputs but implemented the edge cases where words are larger than the line width or empty and a new functionality that cuts the word if its length is larger than the line. P2 did not create a class but three individual functions instead. In this case, our script was not able to parse the messages received from ChatGPT and extract the generated code. Thus, P2 had to struggle a bit until he was able to understand the automation. The final code was clean, with only assertions per test function, but it contained one redundant assertion per function and no edge cases. P3 implemented the largest number of test cases, including all valid and invalid cases and specific exceptions: *InvalidLineWidthException*, *CenteringNotPossibleError*, and *SpreadNotPossibleError*. The final code is not completely clean, though, as it includes repeated assertions for one method and more than one test method for the same function to test. P4 implemented all test cases for all functions, but no edge cases and the final code is not correct (see Log inspection, next paragraph). P5 worked with no assistance from the AI and implemented only one test case per function, with valid input and no edge cases. F1 produced one test function with three assertions, each testing a valid input. One valid case was missing and no edge case or invalid value was tested. F1 was the fastest to complete the task.

Log and Screen Inspection. For all developers who collaborated with the AI, we logged the interactions and watched the screens' recordings. The last column of Table 2 reports the number of interactions. At each interaction, we logged the test and production code, the execution trace, and the changes made by the developer to fix the AI output and make the test pass. All the changes have

finally produced a correct code except in the case of P4. ChatGPT recommended these changes at the final interaction:

```
Changes made to the code:
1. In the `test_TwoWordSpreadEven` test, the `TextFormatter`
    variable was mistakenly assigned instead of `textFormatter`.
2. In the `test_TwoWordSpreadOdd` test, the `TextFormatter`
    variable was mistakenly assigned instead of `textFormatter`.
3. In the `test_WordCenterOdd` test, the expected result was
    corrected by removing the extra spaces.
```

However, the proposed changes do not fix the bug but simply avoid the execution of the code revealing the bug. For F1, no test failed, suggesting that the AI was more concerned with not failing the test than with developing a high-quality solution.

Analysing Feedback. We finally asked the participants about their experience with AI. The answers are reported in Table 3. The positive feeling described by P1 is related to the compatibility of the tool with the way P1 works. This feeling of comfort is known to be a key determinant for acceptance and adoption of new technologies and methodologies in software engineering [14]. This aspect will also be important for the adoption of AI-based tools and, as such, should be considered in the development of this new generation of tools. P2, who created a set of functions rather than a class, got frustrated when the GenAI tool did not work as expected. Apparently, the same frustration has also been observed in students in a study on using GenAI tools for teaching software engineering [4].

Table 3. Feedback

	Perceived difficulty	Feedback
P1	Easy	To be honest, the "presence" of the AI made me a little unsure in the beginning, because I was concerned about its behavior and if I should adapt to fit its need. Once I realized the AI would adapt to my needs (in particular my dev-flow), I think the experience went way more smoothly
P2	Easy	The tool did not work as expected. It seemed kinda buggy, as it did not add any code to the existing file. I was expecting more from an AI tool as normally ChatGPT is able to complete such trivial tasks
P3	Easy	It was fun, the requirements are very broad, so maybe the assumptions can vary a bit from person to person
P4	Fine	NA
P5	Hard	NA

In this case, though, it was not the GenAI that did not process the query as expected but rather our script, which was not designed to extract functions from the messages of ChatGPT. Of course, the developer could not distinguish the difference. Clearly, P2 did not read the task's description, which required developing a class, but this gave us the hint to refactor our tool so that it is now also capable of extracting functions from ChatGPT messages. Also P3 had a positive experience in performing the task autonomously and even suggested the authors some refinement of the experiment. Being expert in both TDD and Python the task was not hard, and the resulting code was the most creative. No feedback was obtained from P4 and P5. Overall, we found that ChatGPT can meet the expectations of the developers in assisting in their job, but without replacing developers in terms of creativity and quality of the code. To obtain satisfactory collaboration with developers, AI should be well integrated into the automation of development activities. The solutions generated may be incomplete or buggy, and non-expert developers may not notice this and trust the AI straight away.

4 Related Work

In this section, we present research works on GenAI for software testing and TDD, in particular. Bird *et al.* [2] analyzed forum discussions from early GitHub's Copilot users, collected their impressions on the tool usage, and observed that support in writing unit tests was one of the major benefits. Producing test cases quickly was the major result of a large survey on the usability of AI programming assistants of Liang *et al.* [11]. They reported that finding edge cases in testing was among the major reasons for using AI. On the other hand, verifying AI answers (e.g., to meet software requirements) was the major reason for not using them. Guilherme and Vincenzi [8] used OpenAI API to generate unit tests and concluded that the tool has a good performance in terms of mutation score and code coverage. GPTDroid [12] uses ChatGPT for GUI testing of mobile apps as a Q&A task and was able to achieve higher coverage and greater efficiency in finding bugs. Lahiri *et al.* [10] propose ITDCG, a workflow with Open AI's Codex for interactive test-driven code generation. Tests and code are generated simultaneously, not incrementally and iteratively, as in TDD. No particular mention is made of how to query the AI or the role of the developer. Tian and Chen [15] introduce Test-case-driven Chain of Thought (TCoT), an approach for improving code generation by using the description of the tests in natural language. The results are promising; however, they did not focus on TDD or any iterative testing process. Piya and Sullivan [13] introduced the LLM4TDD framework to incorporate GenAI into TDD. A developer develops within a coding environment that interacts with a GenAI. The developer manually copies, if needed, code and tests and whether the latter ones fail. An evaluation of the framework reached a success rate of 88.5%. The authors also identified best practices to ensure that ChatGPT solves the correct problem and to reduce the effort. Our work follows these recommendations but also provides 1) different ways of interactions between humans and AI, 2) a structure

for the input with predefined prompts to avoid generating unwanted code, 3) a control layer for the collaborative pattern in which the quality of the code is iteratively verified by the developer. The experiment with real developers helped to understand the issue of such collaboration at a fine granularity level.

5 Conclusions

In this work, we defined interaction patterns between developers and GenAIs for the automation of TDD. We conducted an exploratory experiment with practitioners to evaluate the feasibility of our automation and the quality of the produced solutions. Overall, we found that for our experiment and settings, GenAI can be efficiently used in TDD, but it requires supervision on the quality of the code produced. In some cases, it can mislead non-expert developers and propose solutions that change tests rather than the production code, which may remain buggy, to make tests pass. In future work, we will extend our methodology to incorporate other interaction patterns (e.g., the developer can choose freely how to query the GenAI), different automation and GenAI, and involve a larger number of practitioners both in the experiment and in the feedback.

Acknowledgments. We thank the practitioners who participated in the study for their valuable contribution. Moritz Mock is partially funded by the National Recovery and Resilience Plan (Piano Nazionale di Ripresa e Resilienza, PNRR - DM 117/2023).

References

1. Beck, K.: Test Driven Development: By Example. Addison-Wesley (2022)
2. Bird, C., et al.: Taking flight with copilot. Queue **20**(6), 35–57 (2022)
3. Causevic, A., et al.: Factors limiting industrial adoption of test driven development: a systematic review. In: Fourth IEEE International Conference on Software Testing, Verification and Validation, pp. 337–346 (2011)
4. Choudhuri, R., et al.: How far are we? The triumphs and trials of generative ai in learning software engineering. In: ICSE (2024)
5. Ernst, N.A., Bavota, G.: Ai-driven development is here: should you worry? IEEE Softw. **39**(02), 106–110 (2022)
6. Fucci, D., et al.: A dissection of the test-driven development process: does it really matter to test-first or to test-last? IEEE Trans. Softw. Eng. **43**(7), 597–614 (2017)
7. Ghafari, M., et al.: Why research on test-driven development is inconclusive? In: ESEM, pp. 1–10. ACM, New York (2020)
8. Guilherme, V., Vincenzi, A.: An initial investigation of ChatGPT unit test generation capability, pp. 15–24. SAST, New York (2023)
9. Karac, I., Turhan, B.: What do we (really) know about test-driven development? IEEE Softw. **35**(4), 81–85 (2018)
10. Lahiri, S.K., et al.: Interactive code generation via test-driven user-intent formalization (2023)
11. Liang, J.T., Yang, C., Myers, B.A.: A large-scale survey on the usability of AI programming assistants: successes and challenges. In: ICSE, pp. 605–617 (2024)

12. Liu, Z., et al.: Make LLM a testing expert: bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In: ICSE (2024)
13. Piya, S., Sullivan, A.: LLM4TDD: best practices for test driven development using large language models (2023). https://doi.org/10.48550/arXiv.2312.04687
14. Riemenschneider, C.K., Hardgrave, B.C., Davis, F.D.: Explaining software developer acceptance of methodologies: a comparison of five theoretical models. IEEE Trans. Softw. Eng. **28**(12), 1135–1145 (2002)
15. Tian, Z., Chen, J.: Test-case-driven programming understanding in large language models for better code generation (2023). https://doi.org/10.48550/arXiv.2309.16120

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

